Real-Time Adaptive Partition and Resource Allocation for Multi-User End-Cloud Inference Collaboration in Mobile Environment

Yiran Li, Zhen Liu, *Member, IEEE*, Ze Kou, Yannan Wang, Guoqiang Zhang, Yidong Li, *Senior Member, IEEE*, and Yongqi Sun

***Abstract*—The deployment of Deep Neural Networks (DNNs) requires significant computational and storage resources, which is challenging for resource-constrained end devices. To this end, collaborative deep inference is proposed, in which the DNN is divided into two parts and executed on the end device and cloud respectively. The selection of DNN partition point is the key challenge to realize end-cloud collaborative deep inference, especially in mobile environments with unstable networks. In this paper, we propose a Real-time Adaptive Partition (RAP) framework, in which a fast split point decision algorithm is proposed to realize real-time adaptive DNN model partition in the mobile network. A weighted joint optimization of DNN quantization loss, inference and transmission latency is performed. We further propose a Joint Multi-user Model Partition and Resource Allocation (JM-MPRA) algorithm under RAP framework. JM-MPRA aims to guarantee the optimized latency, accuracy and resource utilization in the multi-user scene. Experimental evaluations have demonstrated the effectiveness of RAP with JM-MPRA in improving the performance of real-time end-cloud collaborative inference in both stable and unstable mobile networks. Compared with the state-of-the-art methods, the proposed approaches can achieve up to 5.06x decrease in inference latency and bring performance improvement of 1.52% in inference accuracy.**

***Index Terms*—Edge intelligence, end-cloud collaborative inference, DNN model partition, resource allocation.**

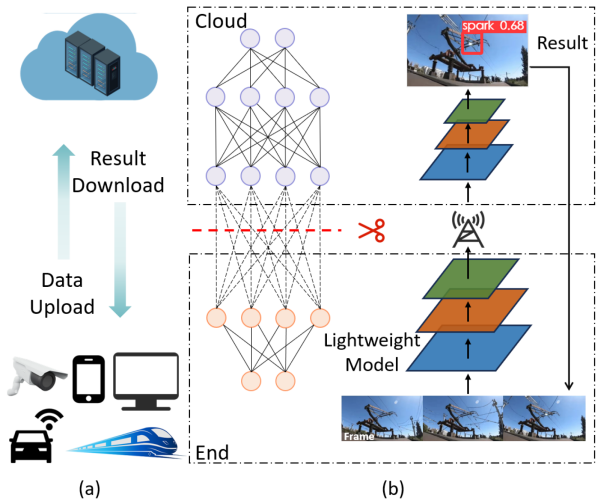


Fig. 1. The architecture of End-Cloud collaborative inference. (a) displays a typical cloud-centric data processing model between End devices and the Cloud. (b) shows a DNN model is split and deployed between End devices and the Cloud. For example, on high-speed trains, an object detection model can be collaboratively deployed and executed between the train-mounted video camera and the cloud to complete a real-time pantograph fault detection task.

I. Introduction

THE rapid development of Deep Neural Networks (DNNs) [1] has stimulated the development of intelligent computer vision [2] applications such as autonomous driving, object detection [3], and augmented reality [4], enabling the solution of complex inference tasks. These intelligent applications require collecting large-scale data on end devices and have high processing requirements for low latency and high accuracy. Usually, the data collected on the end devices is uploaded to the cloud for processing, after which the processing results are transmitted back to these devices, as illustrated in Fig. 1(a). Nonetheless, for certain scenarios and tasks, this cloud-centric approach may face data transmission and processing latency issues, especially in a mobile environment with unstable networks. As a solution, sinking the cloud computing capacity from the core network to end devices is becoming an emerging computing paradigm [5], [6].

However, deploying DNNs on resource-constrained end devices presents significant challenges. Existing research has explored many techniques for efficient DNN inference on such constrained devices, such as model compression [7], [8] and compact network design [9], [10]. These approaches aim to decrease resource requirements by modifying the DNN model

Manuscript received 15 January 2024; revised 12 May 2024; accepted 30 June 2024. Date of publication 17 July 2024; date of current version 5 November 2024. This work was supported in part by the Fundamental Research Funds for the Central Universities under Grant 2022JBZY027 and in part by the National Natural Science Foundation of China under Grant U2268203 and Grant 62262018. Recommended for acceptance by J. Ko. *(Corresponding author: Zhen Liu.)*

Yiran Li, Zhen Liu, Ze Kou, and Yannan Wang are with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing 100044, China, and also with the Engineering Research Center of Network Management Technology for High Speed Railway, Ministry of Education, Beijing 100044, China (e-mail: liyiran@bjtu.edu.cn; zhliu@bjtu.edu.cn; kouzkz@bjtu.edu.cn; yannanwang@bjtu.edu.cn).

Guoqiang Zhang is with the School of Information and Technology, Hainan Normal University, Hainan 571158, China (e-mail: zgqoop_cn@hotmail.com).

Yidong Li and Yongqi Sun are with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing 100044, China, and also with the Key Laboratory of Big Data and Artificial Intelligence in Transportation Ministry of Education, Beijing 100044, China (e-mail: ydli@bjtu.edu.cn; yqsun@bjtu.edu.cn).

Digital Object Identifier 10.1109/TMC.2024.3430103

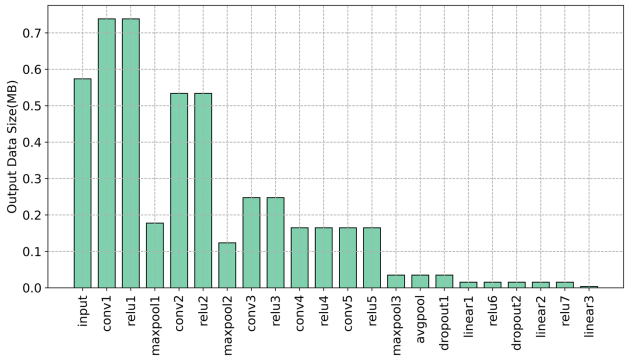


Fig. 2. Output data size of each layer in AlexNet.

structure or reducing the number of parameters. However, they also suffer from a loss of accuracy.

To take full advantage of the local processing capabilities of end devices and the high computing power of the cloud to achieve high accuracy and low latency, model partition [11], [12], [13], [14], [15] has been proposed in recent years. Typically, the full DNN model is deployed on end device and cloud server. Based on the DNN model partition method, DNN model will be split at appropriate split layer points and partially executed on end device and the cloud respectively. As shown in Fig. 1(b), a typical End-Cloud(EC) architecture consists of 2 layers: the end devices (e.g., smartphone, IoT devices) and the cloud. Compared to the typical cloud-centric inference architecture, EC architecture has several advantages. First, end devices are closer to the data source, allowing data pre-processing so as to preserve data privacy. Second, communication latency caused by uploading large amounts of raw input data to the cloud can be avoided.

Fig. 2 displays the output data size of each layer in AlexNet. It can be observed that the output data size at some intermediate layers is significantly smaller than the raw input data. In addition, in discriminative DNN models utilized for recognition applications, the layer size gradually decreases from the input to the output [16]. In this paper, we mainly focus on discriminative DNN model. Based on this observation, when the DNN model partition is performed, only the intermediate results output by the end device will be sent to the cloud, thus reducing communication costs and end-cloud latency.

While significant advancements have been made in this field, prior work mainly focused on finding optimal partitioning schemes in a specific network environment, ignoring the real-time model partition adaptive to mobile unstable networks. Factors such as interference and signal propagation delays in mobile networks lead to unstable network bandwidth. In addition, existing model partition methods are usually studied for single-user (In this paper, user and end device have the same meaning). Actually, there are many multi-user end-cloud scenarios, such as mobile vehicles in the Internet of Vehicles environment, various devices in high-speed trains, and etc. These end devices compete with each other when they ask for collaborative inference with the cloud. This leads to the fact that the communication and computing resources of the cloud will be not enough to meet the needs of multi-user devices, thus affecting the performance of collaborative inference. Consequently, it is also necessary to discuss the issue of multi-user DNN model partition.

In this paper, we propose a Real-time Adaptive Partition (RAP) framework, which can enable real-time collaborative inference over the cloud and end devices, adapting to unstable networks. In this framework, local inference latency in end device, transmission latency and quantization loss are jointly considered to select the optimal split point. Additionally, we propose a method to optimize the decision space and reduce decision latency. Furthermore, considering the cloud resource competition among multi-user end-devices, a joint multi-user model and resource allocation (JM-MPRA) algorithm is proposed to jointly and iteratively optimize the cloud's computing resources, communication resources and DNN partition point decision. Our major contributions are summarized as follows:

- We propose the RAP framework for real-time adaptive end-cloud DNN collaborative inference in mobile unstable networks. RAP designs a combing fitting function to perform joint weighted optimization of latency and loss, in which the weights vary with the network quality. Through the Optimal Split Point Searching(OSPS) algorithm, RAP realizes the online decision on the optimal split points for the DNN model partition with the time complexity of $\mathcal{O}(N)$ (N is the number of DNN layers), which greatly reduces the latency in making and remaking decisions in unstable networks.
- Based on RAP framework, we further propose the JM-MPRA algorithm to realize multi-user end-cloud collaborative inference by optimizing resource allocation in cloud. JM-MPRA algorithm jointly and iteratively optimizes the allocation of the resources of cloud and the decision of multi-user DNN model split points. In addition, a Resource Allocation Vector Initialization (RAVI) algorithm is proposed, which can initialize the resource allocation vector by pre-scheduling the computing and communication resource of the cloud, thus reducing the iteration times of JM-MPRA algorithm and improving the efficiency.
- We carry on the experiments on mobile stable networks, in which we use 3G and 4G network bandwidths. We also do the experiments on two mobile unstable networks, which are urban traffic network and railway network. In the experiments, the Raspberry Pi serves as the end device and a GPU server serves as the cloud. The experimental results show that compared to the existing methods, the proposed RAP framework can bring a performance of 3.73x decrease in total inference latency and 1.52% improvement in inference accuracy, respectively. In the scenario of joint multi-user model partition and resource allocation, the proposed RAP with JM-MPRA algorithm can achieve up to 3.24x decrease in total inference latency and improve inference accuracy by 0.54% compared to other methods.

II. Related Work

Executing DNN models directly on mobile end devices is challenging due to limited resources. A large number of attempts

have been made to optimize DNN models computation on end devices.

A. Model Compression

Model compression is achieved by reducing the model parameters or simplifying the model structure to reduce computational and storage resources. Model quantization is a technique for converting floating-point precision to low-bit precision, which can effectively reduce model computation intensity, parameter size and memory consumption. Post-Training Quantization (PTQ) [17], [18], [19] refers to directly quantifying weights and activations after model training is completed, without the need for retraining the model. However, this method is very sensitive to abnormal values, because the abnormal values may lead to excessive rounding errors, then affects the accuracy of quantitative models. While Quantization-Aware Training(QAT) [17], [20], [21] performs quantitative operations during training and optimizes model parameters through backpropagation. Fake-quantization is introduced during the model training process to simulate the losses caused by the quantization process. In this way, the accuracy loss of the quantized model can be further reduced. There are other methods for model compression, including model pruning [22], [23], knowledge distillation [24], [25] and neural architecture search (NAS) [26]. While the lightweight model after model compression significantly reduces the number of parameters and calculations, deploying it on specific end devices, such as the Raspberry Pi, smart cameras and sensors, remains challenging.

B. End-Cloud Model Partition

Model partition aims at finding the optimal split point with the minimum latency or energy. Then, DNN is split at the split point and is executed on different devices. For example, Neurosurgeon [12] partitions DNNs between mobile devices and the cloud, which dynamically selects the best partition solution to meet the requirement of the inference latency or energy consumption. However, Neurosurgeon handles only chained DNNs rather than graph-structured DNNs. The DADS approach proposed in [11] can correctly partition directed acyclic graph (DAG) DNN models (e.g., GoogLeNet [27] and ResNet [28]), but its time complexity is too high to achieve real-time partition decision on end devices. QDMP [29] converts the model partition of DNNs with DAG topology into the min-cut problem and can solve the problem quickly. JointDNN [16] models the problem of finding the optimal partitioned solution as the shortest path problem. AUTO-SPLIT [30] minimizes the overall latency under an end device memory constraint and a given error constraint, by jointly optimizing the split point and bid-widths of layers on the end device. RT-DMP [31] explores the joint optimization of dynamic DNN model partitioning, processing and network resources by leveraging virtual queue-based Lyapunov optimization framework. Irtija et al. [32] used Satisfaction Games to determine where the optimal portion of a node's task is offloaded. The authors in [33] designs a distributed DNN inference scheme with fine-grained model partition via a multi-task learning approach under specific delay constraint. Although most methods conduct a comprehensive search for split points in stable networks, model partition in dynamic networks still needs further research.

C. End-Cloud Resource Allocation

Resource allocation refers to the allocation and optimization of limited resources to achieve optimal performance in multi-user situations. Dai et al. [34] proposed an end-edge-cloud coordinated computing scheduling method based on deep reinforcement learning, with the goal of minimizing the maximum task waiting time. CORA [35] studied the joint optimization of computing offloading and resource allocation, which can adapt to the dynamic changes of the network. The scheme proposed by Tan et al. [36] aims to optimize task offloading decisions and allocation of computing and communication resources among all users, in order to minimize total energy consumption, computing costs, and maximum latency. Tang et al. [37] proposes an iterative alternative optimization algorithm for computing resource allocation in mobile edge computing environment. Kim et al. [38] designed different algorithms on the user side and CSP side jointly to determine the resource allocation strategy to minimize the total cost. This includes the user deciding whether to offload the task, and the CSP deciding how to allocate server resources based on state such as queue length, task arrival rate, and so on. Li et al. [39] aimed to minimize the total energy consumption by jointly optimizing the task offloading rate and resource allocation to meet the service latency requirements. Although previous studies have explored computing resource allocation among multiple users, there is still a lack of methods to consider multiple types of resources at the same time and integrate resource allocation.

III. Problem Formulation

In this section, we first introduce some basic setups and the problem formulation of adaptive end-cloud model partition. Then, considering that multi-user end devices with resource allocation in cloud, we further present a detailed problem formulation to the multi-user scenarios. For a clear demonstration, the summary of notations used in the following sections are given in Table I.

A. Adaptive End-Cloud Model Partition

In the problem of adaptive end-cloud model partition, we aim to minimize the weighted latency and quantization loss through adaptive joint optimization that varies with the network quality. Due to the significant gap in computing power between end devices and the cloud, only the local inference latency of the DNN model on the end device is considered, while the cloud inference latency is ignored. Thus we formulate the end-cloud model partition problem as a nonlinear integer optimization problem(INLP).

Assume that a DNN model consists of $N(N \in \mathbb{Z})$ layers, where $\mathbb{Z}$ is a set of non-negative integers. In this section, we only consider model partition for one end device.

TABLE I
SUMMARY OF NOTATIONS

Notations	Descriptions
U	Number of user devices
N	Number of DNN model layers
B	Total communication resource of cloud
C	Total computing resource of cloud
u_i	The i-th user device
t_k^{end}	Local inference latency of the k-th layer
t_k^{trans}	Transmission latency of the k-th layer
l_k^{quant}	Quantization loss of the k-th layer
d_k^{trans}	Output data size of the k-th layer
c_i^{cloud}	Computation of u_i offloading to cloud
t_i^{cloud}	Cloud inference latency of u_i
$w_k(orig)$	Weights of the k-th layer before quantization
$w_k(quant)$	Weights of the k-th layer after quantization
$a_k(orig)$	Activation values of the k-th layer before quantization
$a_k(quant)$	Activation values of the k-th layer after quantization
E_k^w	Weight quantization loss of the k-th layer
E_k^a	Activation quantization loss of the k-th layer
$\omega(B)$	Weight function about bandwidth
$\theta(\delta)$	The unit step function
α	Smoothing factor
δ_i	Proportion of communication resource allocated to u_i
γ_i	Proportion of computing resource allocated to u_i
$f(x)$	Accumulated local inference latency fitting function
$g(x)$	Intermediate output data size fitting function
$h(x)$	Accumulated quantization loss fitting function

Considering limited resources in end device, we deploy the quantified DNN model on the end device. The original bit-width DNN is deployed in the cloud. We measure quantization loss by referring to the method in [17]. $w_k(\cdot)$ and $a_k(\cdot)$ can be denoted as weight and activation for a given bit-width at the DNN k-th layer. Then by using the Mean Square Error (MSE) function, the quantization loss at the k-th layer for weight and activation can be denoted by $E_k^w = MSE(w_k(orig), w_k(quant))$ and $E_k^a = MSE(a_k(orig), a_k(quant))$, respectively. $w_k(orig)$ and $w_k(quant)$ represent the weights of the DNN k-th layer before and after quantization, and $a_k(orig)$ and $a_k(quant)$ represent the activation values of the DNN k-th layer before and after quantization, respectively.

Definition 1: The loss of weight and activation at the DNN k-th layer is $l_k^{quant} = E_k^w + E_k^a$. Then $\sum_{k=0}^{x} l_k^{quant}$ is the accumulated quantization loss from the input layer to the x-th layer.

Definition 2: The local inference latency for executing the DNN k-th layer on the end device is t_k^{end}. Then $\sum_{k=0}^{x} t_k^{end}$ represents the accumulated local inference latency from the input layer to the x-th layer.

Definition 3: The output data transmission latency is t_x^{trans}. The latency happens when transmitting the output data of the x-th layer from the end device to the cloud. $t_x^{trans} = d_x^{trans}/B$, where d_x^{trans} denotes the data output size of the x-th layer and B denotes the bandwidth of the current network.

The 0-th layer represents the input layer. If all layers are executed on cloud or on end, i.e., $x = 0$ or $x = N$, then $t_0^{trans} = d_0^{trans}/B$ or $t_N^{trans} = 0$.

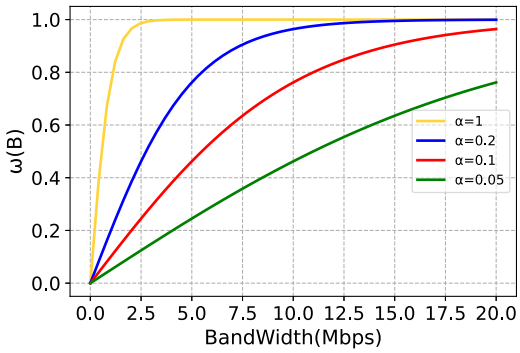


Fig. 3. $\omega(B)$ function.

1) INLP Problem Formulation: Suppose that DNN is split at layer $x, 0 \le x \le N$. The input layer to the x-th layer of DNN are executed on the end device. The remaining layers of DNN are executed in the cloud. Then we can define the objective function as follows:

$$\mathcal{L}(x) = \sum_{k=0}^{x} t_k^{end} + t_x^{trans} + \sum_{k=0}^{x} l_k^{quant}$$
$$\text{s.t. } 0 \le x \le N \tag{1}$$

The objective function of latency and loss is related to the x-th layer, so it can be formulated as $\mathcal{L}(x)$, where the independent variable x is the split layer of DNN. Basically, we can model $\mathcal{L}(x)$ as:

$$\mathcal{L}(x) = t^{end}(x) + t^{trans}(x) + l^{quant}(x) \tag{2}$$

To simplify the calculation, we normalize the latency and loss by the min-max method to make them of the same numerical magnitude, which is defined as:

$$X_{normal} = \frac{X - X_{min}}{X_{max} - X_{min}} \tag{3}$$

where X represents the sum value of t^{end} and t^{trans}, and the value of l^{quant} before normalization respectively. X_{min} and X_{max} represent the smallest and the biggest values of the sum of t^{end} and t^{trans}, as well as the smallest and biggest values of l^{quant}, respectively. X_{normal} represents the normalized value, which is between 0 and 1.

2) Joint Optimization of Latency and Quantization Loss: Taking into account that users' demands on service quality may vary with network quality, we conduct a joint optimization according to bandwidth. When network quality is poor, users have higher requirements for low latency instead of less quantization loss. When the network quality is good, transmission latency is no longer the main factor affecting service quality, higher inference accuracy is more important to users. Thus, we weigh the latency and loss in the objective function $\mathcal{L}(x)$ to achieve adaptive optimization. Specifically, we express the weights of latency and loss using the function $\omega(B)$ with bandwidth B as the independent variable, where $\omega(B) = \frac{e^{\alpha B} - e^{-\alpha B}}{e^{\alpha B} + e^{-\alpha B}}$, as shown in Fig. 3.

The function $\omega(B)$ increases with increasing bandwidth B. The value of $\omega(B)$ is between 0 and 1. $1-\omega(B)$ and $\omega(B)$ are used as the weights of latency and loss respectively, which can make the proportion of latency and loss in the objective function change with bandwidth. α controls the smoothness of $\omega(B)$, which in turn affects the weights of latency and loss. When the bandwidth is small, i.e., the network quality is poor, latency is supposed to be the main optimization objective. Therefore, the weight of latency in the objective function $\mathcal{L}(x)$ should be larger than loss. While when the bandwidth is larger, the weight of quantization loss should be larger. In this way, the optimization prefers to reduce the loss to increase the inference accuracy.

In Fig. 3, we can see that selecting a large α causes the weight of quantization loss in the objective function to increase rapidly at low bandwidth and then approach the stable value. In this way, when the bandwidth is low, latency is not the main consideration of the objective function. When α takes a smaller value, such as 0.05, the quantization loss gradually becomes the main factor of the objective function after about 11 mbps. However, in our experimental process, we found that the optimal split decision for most models tends to offload all data to the cloud when the bandwidth is between 10 mbps and 14 mbps, and considering the loss value as the main factor after 11 mbps may lead to inaccurate decisions. Detailed experimental results are presented in Section VI-C. Therefore, in the experiment, we take α as 0.1. By the function $\omega(B)$, we realize that the optimization priority of latency and loss changes adaptively with the change of bandwidth. $\mathcal{L}(x)$ can be rewritten as:

$$\mathcal{L}_B(x) = (1-\omega(B))\left(t^{end}(x) + t^{trans}(x)\right) + \omega(B) l^{quant}(x) \tag{4}$$

In order to prevent excessive loss, we restrict the maximum precision loss l_{max}^{quant} during optimization. To summarize, the problem can be formulated based on the objective function $\mathcal{L}_B(x)$ along with constraints as follows:

$$\begin{aligned} \min_{x} \ & \mathcal{L}_B(x) \\ \text{s.t. } & l^{quant}(x) \le l_{max}^{quant} \\ & 0 \le x \le N, x \in \mathbb{Z} \end{aligned} \tag{5}$$

B. Adaptive Multi-User End-Cloud Model Partition and Resource Allocation

In this section, we consider the model partition and collaborative inference for multi-user devices, in order to allocate resources reasonably and reduce the overall total inference latency as much as possible. u_i represents the i-th user device, and user device is synonymous with end device. Based on the above section, we jointly formulate the multi-user model partition and resource allocation problem as follows:

As shown in Fig. 4, consider each u_i is configured with a N_i layers pre-trained DNN model. δ_i and γ_i respectively represent the proportion of cloud communication resource and computing resource allocated to u_i, where $\sum_{i=1}^{U}\delta_i = 1$ and $\sum_{i=1}^{U}\gamma_i = 1$.

Definition 4: $t_{i,k}^{trans} = d_{i,k}/(\delta_i \cdot B)$ represents the data transmission latency of the k-th layer on u_i, where $d_{i,k}$ represents the transmission data size of k-th layer on u_i. Considering the multi-user scenario, we update Definition 3 with the δ_i factor. $\delta_i \cdot B$ represents the cloud communication resources allocated to u_i, where B represents the total communication resources of the cloud.

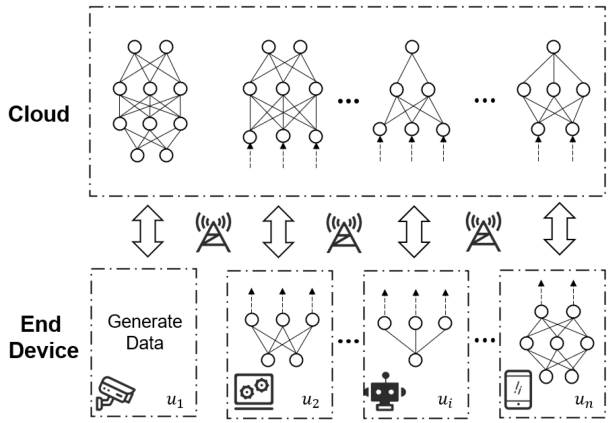


Fig. 4. Multi-user model partition.

Definition 5: $t_i^{cloud} = c_i^{cloud}/(\gamma_i \cdot C)$ represents the cloud inference latency of u_i offloading the model to the cloud, where c_i^{cloud} represents the amount of computation that the device u_i offloads to the cloud. And $\gamma_i \cdot C$ represents the cloud computing resources allocated to u_i, where C represents the total computing resources of the cloud.

In the multi-user scenario, each u_i first obtains the optimal partition decision based on the Adaptive End-cloud Model Partition. After the model split point decision, the latency of the DNN on u_i consists of three parts: Definitions 2, 4, and 5. Thus the latency of a single u_i is formulated as follows:

$$T_i(k_i, \delta_i, \gamma_i) = \sum_{j=0}^{k_i} t_{i,j}^{end} + \frac{d_{i,k_i}^{trans}}{\delta_i \cdot B} + \frac{c_i^{cloud}}{\gamma_i \cdot C} \tag{6}$$

With limited resources, simply finding the optimal split point for each u_i does not effectively reduce the overall latency. Devices with maximum latency affect the global latency. As a result, we define the optimization goal of the multi-user end-cloud model partition problem as minimizing the maximum total latency among all user devices, and making all devices complete the inference tasks as soon as possible in the shortest time. Given all the user device's decision-making model split point k_i, communication resource allocation scheme, and computing resource allocation scheme, we need to find the u_i with the largest latency and define its latency as:

$$UM(K, \Delta, \Gamma) = \max_{i \in U} T_i(k_i, \delta_i, \gamma_i) \tag{7}$$

where K is the decision vector of all users' model split points, Δ is the communication resource vector allocated by the cloud for users, and Γ is the computing resource vector allocated by the cloud for users. Optimizing the largest latency among all users by jointly adjusting the three decision vectors can reduce the overall latency, and improve the inference performance. Finally,

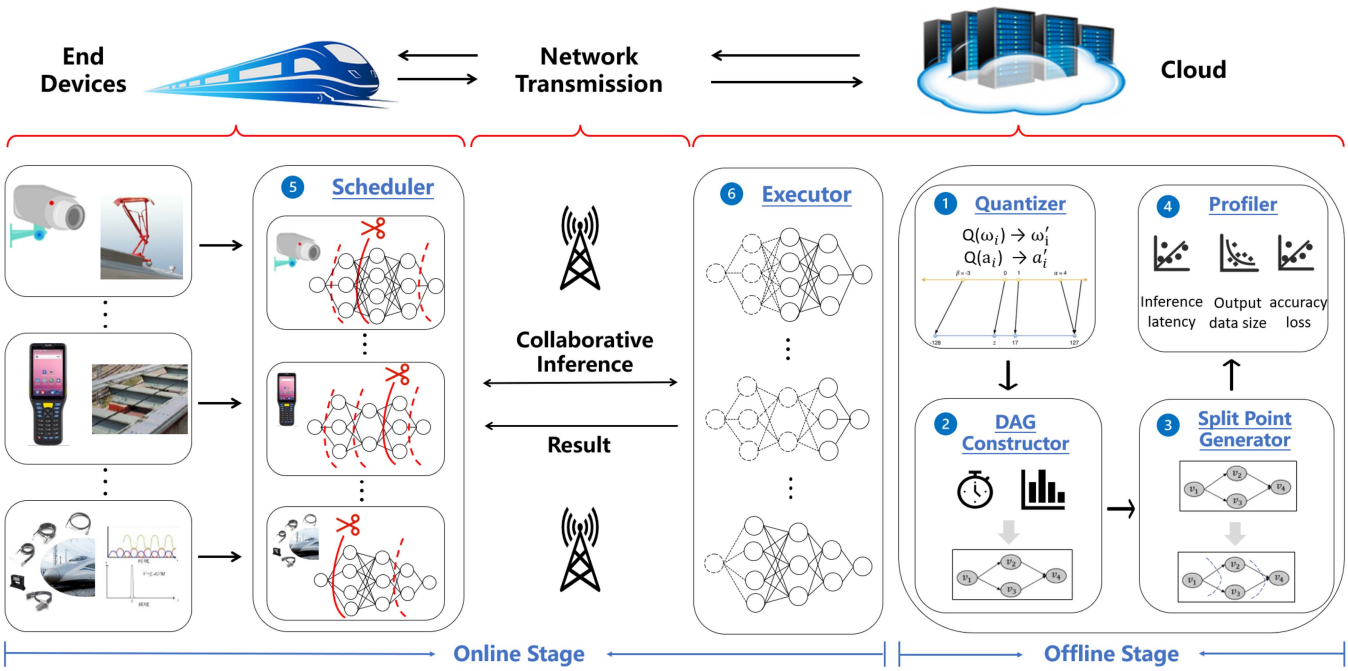


Fig. 5. Real-time Adaptive Partition (RAP) framework. RAP framework includes two stages: offline stage and online stage. In the offline stage, the local inference latency and intermediate data output of each layer of the DNN model are obtained on the end devices at first, and then Quantizer, DAG Constructor, Split Point Generator and Profiler are carried out in the cloud. In the online stage, Scheduler is performed on the end devices to obtain the optimal split point. Executor transmits the optimal split point result to the cloud for collaborative inference.

the problem is formulated as follows:

$$\begin{aligned} &\min\ UM(K,\Delta,\Gamma) \\ &\text{s.t.}\ \sum_{i=1}^{U}\delta_i = 1, \sum_{i=1}^{U}\gamma_i = 1 \\ &\quad 0 \le k_i \le N_i, k_i \in \mathbb{Z} \end{aligned} \tag{8}$$

where the constraints of $\sum_{i=1}^{U}\delta_i = 1$ and $\sum_{i=1}^{U}\gamma_i = 1$ ensure that the communication resources and computing resources of cloud are allocated to all users in proportion, and all the communication and computing resources of cloud are fully utilized. $0 \le k_i \le N_i$ constrains the split points of model partition to ensure the rationality of the split points. This problem is essentially a nonlinear integer programming problem, and involves optimization of multiple decision variabless. It is an NP-hard problem with a complex solution. We will propose methods to optimize the solution of this problem in the next section.

IV. Real-Time Adaptive Partition Framework

To remedy the limitations of existing studies, we propose a Real-time Adaptive Partition (RAP) deep inference framework for end-cloud inference collaboration. RAP optimizes model partition and accelerates decision-making to satisfy the latency requirements in unstable mobile networks while considering the accuracy of inference.

A. Overview

RAP works in two stages: offline stage and online stage as shown in Fig. 5.

Offline Stage: At the beginning of the offline stage, RAP quantizes a pre-trained DNN model by the quantizer. The quantized DNN model then will be constructed as a directed acyclic graph (DAG). In the DAG, the split point generator can find a set of potential split points for the model. Finally, local inference latency, output data size, and quantization loss of each layer are calculated and fitted to the continuous domain on the set of potential split points. These tasks can be processed offline in advance in the cloud. Considering the real-time conditions, the initial local inference latency and intermediate data output size of each DNN layer are measured on the end devices. The data obtained in the offline stage will be used as the input of the online stage.

Online Stage: In the online stage, the scheduler obtains the data generated in the offline stage for online optimal split point selection. Then, the executor divides the inference process of DNN at the optimal split point and performs end-cloud collaborative inference. Online optimization is executed on the end device because the end device can directly perceive the current network quality, and can quickly optimize the solution according to the bandwidth. There is no need to upload the current network bandwidth to the cloud for optimization, which will bring excessive online latency. In the multi-user scenario, the decision will be made in the cloud because it involves the information aggregation of multi-user devices and the adjustment of resource allocation.

B. Quantizer

Given a pre-trained DNN, the quantizer uses the method [17] to quantize the weights and activations of the original DNN from 32-bit to 8-bit, which inevitably leads to a loss of accuracy. To evaluate the difference of the weight and activation values before and after quantization, as mentioned in Section III-A, the quantizer uses the Mean Square Error (MSE) function as quantization loss metrics. The quantization scheme is an affine mapping of real numbers r to integers q, as shown in the formula:

$$q = round(r/S) + Z_p \tag{9}$$

where q represents the quantized fixed-point number, r represents the floating-point number before quantization, and S represents the scale factor. We use Z_p to represent the fixed-point number corresponding to floating-point 0 after the mapping of floating-point numbers to fixed-point numbers. Generally, symmetric quantization is used for weight quantization, and asymmetric quantization is used for activated quantization. This is because for weights, they are generally symmetrical, while for activations, their values may be distributed on one side.

During the quantization process in (9), mapping floating-point numbers to integers typically involves rounding operations. Different rounding strategies, such as rounding down and rounding up, can lead to varying degrees of loss. In addition, when quantizing larger 32-bit values to 8-bit, overflow may occur, i.e., the mapped value exceeds the maximum value that 8-bit can represent. More importantly, the process of DNN model inference can lead to accumulation of quantization losses, especially in deep networks where losses may be further amplified at each layer of the network. Although there is no direct mathematical relationship between cumulative quantization loss and inference accuracy, the more quantization loss leads to a higher decrease in accuracy.

To reduce the loss caused by quantization, we only deploy the quantized model on the end device and deploy the original model on the cloud. The more layers executed on the end device, the higher accumulated loss caused by quantization. Therefore, the loss depends on the number of DNN layers running on the end device, which has an important impact on the final split point selection.

C. DAG Constructor and Split Point Generator

After deriving the quantization model from the original DNN model, RAP aims to collect a set of potential split points by analyzing the output data size of each layer.

DAG Constructor: The output of each layer in the DNN model is fed to the subsequent layer. In order to find the set of potential split points in DNN, RAP first constructs the DNN as a directed acyclic graph (DAG), as shown in Fig. 6. For a chain topology DNN, such as AlexNet and VGG-16, all layers are connected in a one-to-one structure. For a graph-structured DNN model, with multiple layers and multi-branches, such as ResNet-50 and GoogLeNet, each layer may have multiple data input or output layers. Generally, given a DNN model, we construct a DAG $G = \langle V, E \rangle$ to represent it. Each vertex $v_i \in V$ denotes a layer of DNN. A directed edge $e_i = \langle v_i, v_j \rangle \in E$ represents adjacent layers, and v_j takes the output of v_i as its input. Note that we take the input layer as v_0, and the value d_0 of the output size of v_0 is the raw input data size.

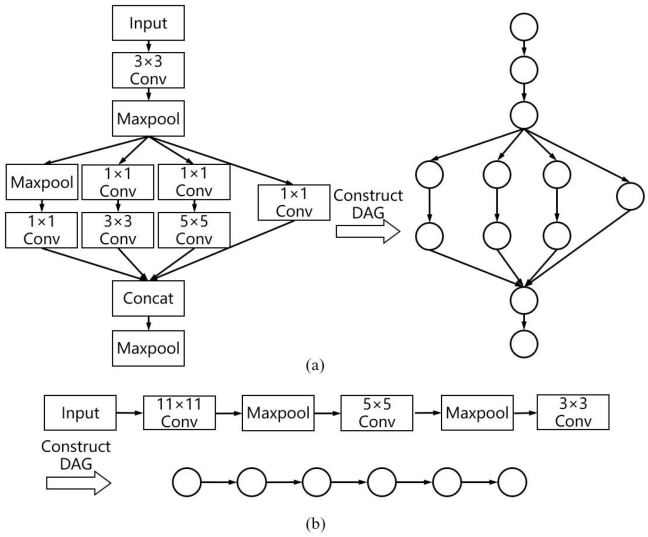


Fig. 6. Constructing the directed acyclic graph (DAG) for the DNN model. (a) constructs a graph-structured DNN model as DAG. (b) constructs a chain topology DNN model as DAG.

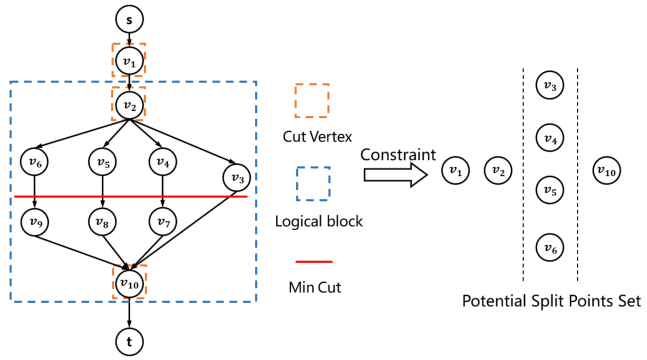


Fig. 7. Min-cut search method first searches all cut vertices shown by the orange box. Then within the logical block, the smallest data output size is found shown by the red line. The cut vertices and the vertices of min-cut within logical blocks whose output data sizes satisfy the constraint are added to the set of split points.

Split Point Generator: According to the existing studies, the redundancy of the set of potential split points leads to a large search space. Therefore, we propose a two-stage potential split point searching method.

1) Cut Vertices: For a graph-structured DNN, in the first stage, we search all the cut vertices in the DAG. The cut vertex is defined as a vertex in the DAG whose removal makes the DAG no longer connected. As shown in Fig. 7, the vertices v_1, v_2, v_{10} in the orange box are cut vertices. If any one of the three cut vertices is removed, the DAG will be divided into two parts and no path connects the input vertex s to the output vertex t.

2) Min-Cut Vertices: There are generally several multi-branches in the DNN model, such as the residual block in ResNet, called logical blocks, as shown in the blue box in Fig. 7. If the model is split within a logical block, multiple layers need

to be split. Therefore, in the second stage, we use the min-cut method in graph theory to select a set of layers with the smallest sum of intermediate data output in the logic block for model partition. A min-cut example is shown as the red line in Fig. 7, and the edges cut by the red line have the minimum weight sum. The vertices v_3, v_4, v_5, v_6 connected to these edges inside the logic block are referred to as min-cut vertices.

For a graph-structured DNN, its potential split points set consists of multiple cut vertices and multiple min-cut vertices. All cut vertices constitute a cut vertex set defined as V_{cut}, and all the min-cut vertex constitute a min-cut vertex set defined as V_{mincut}. For a chain topology DNN, its potential split points set only consists of cut vertices.

In the following, we analyze the conditions that the potential split points need to meet.

Lemma 1: The point in V_{cut} whose output data size meets $d_i < d_0$ or the points in V_{mincut} meet $\sum_{i=1}^{n} d_i < d_0$, where $\sum_{i=1}^{n} d_i$ is the minimum sum of output data sizes of n vertices within a logical block can be added to the potential split points set. Then, the potential split points set defined as $\mathcal{P} = \{v_i | i \in 0, 1, \ldots, n | \sum_{i=1}^{n} d_i < d_0\}$.

Proof: Assume that the split point is v_j. The output data size of v_j is denoted by d_j, and $d_j > d_0$. Then the total latency is $T_j = \sum_{k=0}^{j} t_k^{end} + d_j/B$. While the total latency for transmitting the raw input data and executing DNN on the cloud is calculated as $T_0 = d_0/B$. Since $d_j > d_0$, then $T_j > T_0$, the layer whose output data size is larger than the original input data can't be the potential optimal split point for the DNN partition. The same can be proven by using the minimum cut as the split point. ■

The Lemma 1 ensures that the output data size at the split point is smaller than the raw input data size. Otherwise, transmitting the raw input data and executing DNN in the cloud is better.

Lemma 2: The consecutive split points with the same output data size are not in $\mathcal{P}$.

Proof: For example, usually, Convolutional (CONV) layer is followed by Batch-Normalization (BN) layer. If they have the same output data size d_m, and have $d_m < d_0$. Then as Lemma 1, the two layers are the potential split points in $\mathcal{P}$. Let the CONV layer be denoted as v_m, BN layer be denoted as v_{m+1}. Then the total inference latency for partition in the CONV layer is $T_c = \sum_{k=0}^{m} t_k^{end} + d_m/B$, the latency for partition in the BN layer is $T_b = \sum_{k=0}^{m+1} t_k^{end} + d_m/B$. Obviously, $T_c < T_b$. We should delete the BN layer from $\mathcal{P}$. Therefore, when the output data size is the same, the split point closer to the input layer will have less accumulated local inference latency and accumulated loss on end devices. ■

The Lemma 2 ensures fewer potential split points in P, resulting in a smaller search space for online optimization decision-making, which is beneficial for decision acceleration.

D. Profiler

As we mentioned in Section IV-A, considering the real-time conditions, the initial local inference latency and intermediate data output size of each DNN layer are measured on the end devices. However, the time complexity of solving nonlinear integer programming problems, as the function (5), and the corresponding decision time are unacceptable for the low latency requirements on the end device. In order to enable the end device to quickly find the optimal split point in the online stage, we first relax the integer constraint on the split layer in function (5) to the real number field. Accordingly, profiler fits the discrete data of each layer to the continuous domain. As shown in Fig. 8, in the feasible split points set $\mathcal{P}$ of VGG-16 model, the accumulated local inference latency $f(x)$ and the accumulated loss $h(x)$ usually increase linearly and the output data size $g(x)$ usually decreases exponentially along with x. The data of potential split point sets of other models used in our experiment also show similar rules. Based on these observations, we assume $f(x)$ and $h(x)$ are increasing linear functions and $g(x)$ is a decreasing exponential function. Therefore, the objective function (5) can be rewritten as:

$$\mathcal{L}_{\mathcal{B}}(x) = (1 - \omega(B))(f(x) + g(x)) + \omega(B)h(x) \tag{10}$$

Theorem 1: The end-cloud model partition problem can be solved as a convex optimization problem.

Proof: Obviously, after relaxing the split point into a continuous space, the feasible domain of function (10) is a convex set. According to the properties of linear and exponential functions, f(x), g(x) and h(x) are all convex functions. As for the function $\omega(B)$ defined in Section III-A, its value is independent of the split point and is determined with different bandwidths in the mobile environment. Thus, in the objective function (10), $1 - \omega(B)$ and $\omega(B)$ are both constants greater than 0. Furthermore, from the properties of convex functions, it is known that a convex function multiplied by a constant is still convex. Therefore, $\mathcal{L}_{\mathcal{B}}(x)$ is a convex function. In this way, the nonlinear integer programming problems in Section III-A can be transformed into convex optimization problems. As for the constraints:

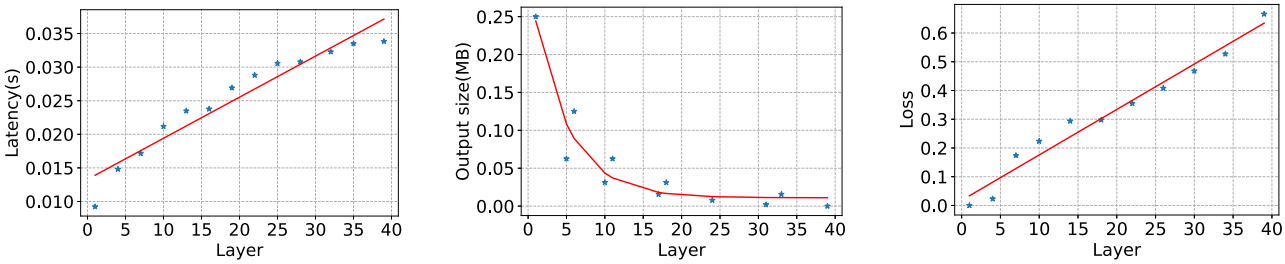
$$h(x) \leq l_{max}^{quant}, 0 \leq x \leq N$$

There is always a feasible solution that strictly holds the above inequalities. Consequently, the strong duality holds since the convex optimization problem satisfies the Slater's condition. From the strong duality, it can be concluded that the optimal solution of the original problem is the solution of the dual problem which can be obtained according to KKT conditions.■

E. Scheduler

To make fast decisions on the optimal split point when the network quality is unstable, we propose an Optimal Split Point Searching(OSPS) algorithm to efficiently find the optimal split point, as Algorithm 1 shows.

As shown in Section IV-D, the input required for Algorithm 1 can be obtained in advance from the offline stage. In Algorithm 1, the end device first finds the optimal solution x of the objective function $\mathcal{L}_{\mathcal{B}}$ (10) on the continuous domain based on the bandwidth, using the method in [40]. Then the result is extended to the discrete domain to consider the integer parts $\lfloor x \rfloor$ and $\lceil x \rceil$. If $\mathcal{L}_{\mathcal{B}}(\lfloor x \rfloor) \leq \mathcal{L}_{\mathcal{B}}(\lceil x \rceil)$, the potential optimal solution $\mathcal{X} = \lfloor x \rfloor$, otherwise $\mathcal{X} = \lceil x \rceil$. Because the result is migrated



(a) Accumulated local inference latency fitting (b) Output data size fitting (c) Accumulated loss fitting

Fig. 8. The accumulated local inference latency function f, intermediate output data size function g and accumulated quantization loss function h of potential split points in the P of VGG-16 model are analyzed and fitted to the continuous domain.

Algorithm 1: Optimal Split Point Searching (OSPS) Algorithm.

Input:
$f(x)$: The accumulated local inference latency;
$g(x)$: The intermediate output data size;
$h(x)$: The accumulated loss;
B: Bandwidth;
Output:
$\mathcal{X}$: The best split point on the $\mathcal{X}$-th layer;

$\mathcal{L}_{\mathcal{B}}(x) = (1-\omega(B))(f(x)+g(x)) + \omega(B)h(x)$;
$x = minimize(\mathcal{L}_{\mathcal{B}}(x))$;
if $\mathcal{L}_{\mathcal{B}}(\lfloor x \rfloor) \leq \mathcal{L}_{\mathcal{B}}(\lceil x \rceil)$ **then**
  $\mathcal{X} \leftarrow \lfloor x \rfloor$;
end if
if $\mathcal{L}_{\mathcal{B}}(\lfloor x \rfloor) \geq \mathcal{L}_{\mathcal{B}}(\lceil x \rceil)$ **then**
  $\mathcal{X} \leftarrow \lceil x \rceil$;
end if
if $t^{end}(\mathcal{X}) + t^{trans}(\mathcal{X}) > \min(t^{end}(N), t^{trans}(0))$ **then**
  $i \leftarrow \lfloor x \rfloor$;
  $j \leftarrow \lceil x \rceil$;
  while $i \geq 0$ and $j \leq N$ **do**
    if $t^{end}(i) + t^{trans}(i) < \min(t^{end}(N), t^{trans}(0))$ **then**
      $\mathcal{X}^* \leftarrow i$;
    else if $t^{end}(j) + t^{trans}(j) < \min(t^{end}(N), t^{trans}(0))$ **then**
      $\mathcal{X}^* \leftarrow j$;
    else
      $i \leftarrow i-1, j \leftarrow j+1$;
    end if
  end while
  $\mathcal{X} \leftarrow \mathcal{X}^*$;
end if
return $\mathcal{X}$;

from the continuous domain to the discrete domain by rounding, the rounded result may not be optimal.

What's more, considering the impact of quantization loss, it may cause the total latency to be greater than that of End-Only (i.e., perform the entire inference on the end device) and Cloud-Only (i.e., perform the entire inference on the cloud). As a solution, we perform further validation to fine-tune the rounded result. Specifically, we compare the real accumulated inference and transmission latency of layer $\mathcal{X}$ (i.e., $t^{end}(\mathcal{X}) + t^{trans}(\mathcal{X})$) with End-only and Cloud-only (i.e., $\min(t^{end}(N), t^{trans}(0))$). When $t^{end}(\mathcal{X}) + t^{trans}(\mathcal{X}) < \min(t^{end}(N), t^{trans}(0))$, $\mathcal{X}$ is identified as the optimal split point. Because fine-tuning will not make the loss change greatly, we only consider fine-tuning the latency. Otherwise, the Algorithm 1 continues to search the set $\mathcal{P}$ of potential split points from $\mathcal{X}$ to find the optimal split point $\mathcal{X}^*$ which is closest to $\mathcal{X}$ with $t^{end}(\mathcal{X}^*) + t^{trans}(\mathcal{X}^*) < \min(t^{end}(N), t^{trans}(0))$. In the offline stage, the cloud and the end devices interact to obtain the aforementioned data. After obtaining the optimal split point generated by the scheduler, the executor will split the inference task of the model based on that point. Then, the end device executes the first partial quantization model and uploads the intermediate results to the cloud. The cloud uses intermediate results as input to execute the remaining original model.

Theorem 2: Given a DNN model, in the set $\mathcal{P}$ of potential split points, Algorithm 1 can always find a feasible DNN partition with the minimum total latency. And the time complexity of Algorithm 1 is $\mathcal{O}(N)$, where N is the number of DNN layers.

Proof: From Lemma 1, it can be concluded that the potential split points set $\mathcal{P}$ consists of cut points and min-cut points. And the output data size of each of the potential split points is smaller than the raw input data size. Therefore, there may be partition decisions that are better than Cloud-Only, which requires all the raw input data to be uploaded to the cloud with high transmission latency.

Besides, the constraints and fine-tuning in Algorithm 1 can ensure the selected split point outperforms the End-Only or Cloud-Only. Therefore, by searching the potential split points, a feasible partition decision with the minimum total inference latency of DNN inference can always be obtained.

The time complexity of the Algorithm 1 is mainly affected by the traversal search of the optimal split points. According to the potential split points set $\mathcal{P}$ defined in Lemma 1, we have $card(\mathcal{P}) < N$, where N is the number of DNN layers. Therefore, the traversal search of the optimal split points can be completed within the time complexity of $\mathcal{O}(N)$. ■

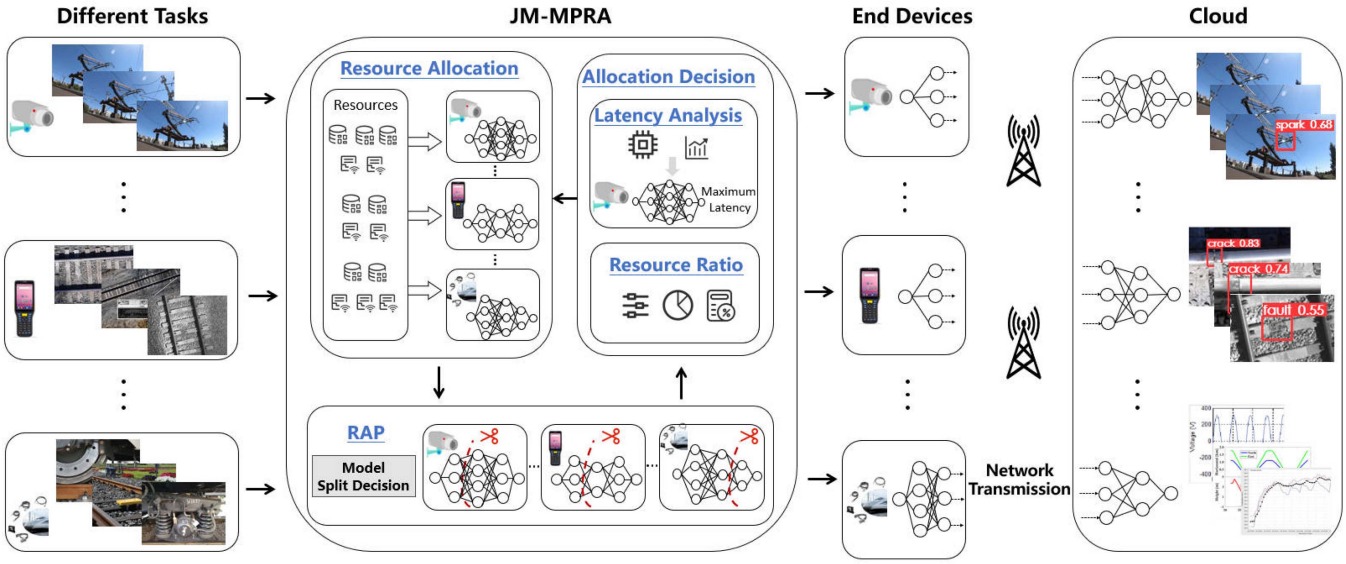


Fig. 9. Collaborative Inference Framework Integrating Joint Multiuser Model Partition and Resource Allocation (JM-MPRA). First, each end device deployed with different pre-trained DNN models performs the partition algorithm of RAP to generate feasible split points. Then, the JM-MPRA algorithm continuously adjusts the decision of model partition and resource allocation to achieve the minimum global latency. Finally, each device executes the previous layers of the pre-trained model and uploads the intermediate data to the cloud. The cloud executes the remaining layers using the resources allocated to each device.

F. Discussion on Decision Time

DADS [11] constructs the entire DNN model as a DAG and adds virtual start and end nodes to convert DAG into a corresponding latency graph. The time complexity of constructing the latency graph is $\mathcal{O}(N+M)$, where N is the number of layers, M is the number of edges in DAG. Then DADS uses the min-cut method to split the whole DNN model based on the latency graph. The time complexity of the whole DADS method is $\mathcal{O}((M+N)N^2)$.

QDMP [29] accelerates the decision speed by finding min-cut in subgraphs. It first finds all the cut vertices and constructs the latency graph for subgraphs between adjacent cut points. The time complexity is $\mathcal{O}(N+M)$. Then QDMP finds the min-cut in each subgraph. Combining the two steps, the overall time complexity of QDMP is $\mathcal{O}((N+M)k^2)$, where k represents the average number of vertices between two consecutive cut vertices.

OSPS algorithm makes DNN split decisions with a linear time complexity $\mathcal{O}(N)$ by traversing the pre-computed potential split points. In contrast, DADS and QDMP need to search all possible points in DAG, whose time complexity is approximately $\mathcal{O}(N^3)$. When the network bandwidth changes frequently in a mobile unstable network, DADS and QDMP need to frequently reconstruct the latency graph due to changes in edge weights. Moreover, when the DNN model becomes complex with the number of layers N becomes large, the decision time of these two methods will increase significantly.

V. Joint Multi-User Model Partition and Resource Allocation

Based on the RAP framework, we extend the problem to multi-user scenarios, aiming to achieve both the optimized model partition of end devices and resource allocation in the cloud.

A. Overview

In the multi-user scenarios, each u_i is deployed with a different pre-training DNN model and performs different inference tasks, as shown in Fig. 9. Each task competitively asks for different computational power and resources in the cloud. Aiming to better utilize resources, minimize overall latency we further propose a Joint Multi-user Model Partition and Resource Allocation (JM-MPRA) algorithm based on the RAP framework, as shown in Algorithm 2.

First, JM-MPRA algorithm will initially allocate computing and communication resources of the cloud equally to all devices. Then the u_i with the worst latency will schedule the resources allocated by the cloud to other user devices. By reallocating the resources of the cloud, the total inference latency of each user device will also be updated. The u_i with the worst latency will continue to compete for resources and the cloud resource allocation will be iteratively adjusted until the adjustment of the resource allocation cannot further improve the overall latency. During this process, the algorithm will also continuously update the partition scheme of each user device.

B. Algorithm Analysis

In Algorithm 2, the offline results of RAP framework will be used as the input of JM-MPRA algorithm in the cloud. JM-MPRA algorithm finally outputs three vectors, which are respectively the model partition scheme of each u_i, the communication resources, and computing resources allocated by the cloud to each u_i. According to these three outputs, each

Algorithm 2: JM-MPRA Algorithm.

Input:
P: Potential split points set;
U: Number of user devices;
$(f(x), g(x), h(x))$: Offline data of model deployed on u_i;
Output:
K: Split decision vector;
Δ: Communication resources allocation vector;
Γ: Computing resources allocation vector;
Initial vector of communication resources and computing resources allocation(Δ_0, Γ_0);
Set resource adjustment step change factor $p = \log_2 U$;
for $s = 0$ to p **do**
  OSPS$(f(x), g(x), h(x), \Delta_0) \rightarrow k_i, T_i(k_i, \delta_i, \gamma_i)$;
  //obtain the optimal split point k_i and latency T_i of u_i
  $T_j^{max} \leftarrow \max_{j \in U} T_j(k_j, \delta_j, \gamma_j)$;
  $j_{max} \leftarrow T_j^{max}$;
  for $u_i, i = 0$ to U **do**
    if $\delta_i > 2^{p-s}$ and $\gamma_i > 2^{p-s}$ **then**
      OSPS$(f(x), g(x), h(x), \delta_i - 2^{p-s}) \rightarrow k_i, T_i(k_i, \delta_i - 2^{p-s}, \gamma_i - 2^{p-s})$;
    else if $\delta_i > 2^{p-s}$ and $\gamma_i < 2^{p-s}$ **then**
      OSPS$(f(x), g(x), h(x), \delta_i - 2^{p-s}) \rightarrow k_i, T_i(k_i, \delta_i - 2^{p-s}, \gamma_i)$;
    else if $\delta_i < 2^{p-s}$ and $\gamma_i > 2^{p-s}$ **then**
      OSPS$(f(x), g(x), h(x), \delta_i) \rightarrow k_i, T_i(k_i, \delta_i, \gamma_i - 2^{p-s})$;
    else
      $m_i = 1$;
      continue;
    end if
    if $T_i > T_j^{max}$ **then**
      $m_i = 1$;
      continue;
    end if
  end for
  $i_{min} \leftarrow T_i^{min}$; //select the i_{min} device with the minimum latency T_i^{min}
  if $T_i^{min} < T_j^{max}$ and $m_i != 1$ **then**
    Scheduling resources according to the judgment:
    $\delta_{i_{min}} \leftarrow \delta_{i_{min}} - 2^{p-s}, \gamma_{i_{min}} \leftarrow \gamma_{i_{min}} - 2^{p-s}$;
    $\delta_{j_{max}} \leftarrow \delta_{j_{max}} + 2^{p-s}, \gamma_{j_{max}} \leftarrow \gamma_{j_{max}} + 2^{p-s}$;
    Update split point decisions of $u_{i_{min}}$ and $u_{j_{max}}$;
    return(K, Δ, Γ);
  end if
end for

DNN model can be divided and deployed. The cloud resource allocation can be carried out.

At the beginning of the Algorithm 2, the resources in the cloud are counted and evenly allocated to each u_i. We define a resource adjustment step change factor $p = \log_2 U$ in the subsequent rescheduling and allocation of resources. U is the number of user devices. The algorithm loops from 0 to p, with a step size of 2^{p-s} for each adjustment of communication resource proportion and computing resource proportion, where s is the loop variable. According to the definition, the step size is large at the beginning, and then reduces continuously in the iteration until the step size is 1. In this way, the algorithm can gradually reduce the step size of resource adjustments, thus speeding up the convergence of the algorithm and ensuring a feasible resource allocation scheme to be found. After that, JM-MPRA will use the proposed OSPS algorithm in RAP to find the optimal split point under the current resource allocation scheme for each u_i.

According to the optimal split point decision, the total latency T_i of each u_i can be obtained. In each iteration, select the device with the maximum latency T_j^{max} from all devices and label it as j_{max}. Then, JM-MPRA compares the communication resource proportion δ_i and the computing resource proportion γ_i of other u_i with the current resource adjustment step size, and determines whether the resources allocated to the u_i can be scheduled. If δ_i and γ_i of u_i do not meet the scheduling conditions, u_i will be marked, indicating that its resources cannot be scheduled under the current resource adjustment step size. On the contrary, if the resource of u_i is schedulable, JM-MPRA will further compute the latest latency of u_i after resource adjustment. If the total latency of u_i is greater than the maximum latency T_j^{max}, then it indicates that the current resource allocation does not reduce the overall latency. Then resource scheduling of u_i can't be performed and u_i will be marked.

At the end of each iteration, if there are unmarked devices, it indicates that resource scheduling can be performed. According to the split decision after resource allocation, the device with the minimum latency after resource scheduling is labeled as i_{min}. Then, the communication and computing resources of $u_{i_{min}}$ are allocated to $u_{j_{max}}$, and the model split points of $u_{i_{min}}$ and $u_{j_{max}}$ after resource adjustment are updated. Nevertheless, If all devices are marked and the resource adjustment step size does not reach the minimum step size, that is, $s = p$, JM-MPRA algorithm will reallocate resources with a smaller resource adjustment step size. When the current resource adjustment step size has reached the minimum step size, it means that any resource adjustment can not reduce the overall latency. Then the current model split point decision scheme and resource allocation scheme are returned as the optimal scheme, and the algorithm ends.

C. Time Complexity Analysis

The time complexity of the JM-MPRA algorithm is mainly affected by the search process for a feasible resource allocation scheme, i.e., the for loop in the inner layer. The maximum iteration number of JM-MPRA algorithm is p. In addition, the JM-MPRA algorithm includes using OSPS algorithm to find the optimal split point for each u_i, which can be completed within the time complexity of $\mathcal{O}(N)$, where N is the number of DNN model layers.

In summary, the time complexity of JM-MPRA algorithm is $\mathcal{O}(p \cdot U \cdot N)$. It is fixed and not too large, therefore, this fixed variable-amount problem can be solved in polynomial time. Besides, with the number of user devices increasing, the time cost of finding the optimal split point for all users is increasing.

Algorithm 3: RAVI Algorithm.

Input:

λ: Threshold value;
μ_B, μ_C: Resource adjustment step size;
(Δ^*, Γ^*): Average allocated resource vector;
$(f(x), g(x), h(x))$: Offline data of model deployed on u_i;

Output:

Initial resources allocation vector (Δ_0, Γ_0);

OSPS$(f(x), g(x), h(x), \Delta^*) \rightarrow T^{arr}(\Delta^*, \Gamma^*)$;
while true **do**
  $i_{min} \leftarrow \min_{i \in U} T^{arr}$;
  $i_{max} \leftarrow \max_{i \in U} T^{arr}$;
  if $\max_{i \in U} T^{arr} - \min_{i \in U} T^{arr} < \lambda$ or $\Delta^*_{i_{min}} \leq \mu_B$ or $\Gamma^*_{i_{min}} \leq \mu_C$ **then**
    return (Δ_0, Γ_0);
  end if
  Resource schedule:
  $\Delta^*_{i_{min}} \leftarrow \Delta^*_{i_{min}} - \mu_B, \Gamma^*_{i_{min}} \leftarrow \Gamma^*_{i_{min}} - \mu_C$;
  $\Delta^*_{i_{max}} \leftarrow \Delta^*_{i_{max}} + \mu_B, \Gamma^*_{i_{max}} \leftarrow \Gamma^*_{i_{max}} + \mu_C$;
  OSPS$(f(x), g(x), h(x), \Delta^*) \rightarrow T^{arr}(\Delta^*, \Gamma^*)$;
end while

As discussed in Section IV, the time complexity of QDMP and DADS for one user is $\mathcal{O}(N^3)$ instead of $\mathcal{O}(N)$. Therefore, the time complexity of JM-MPRA algorithm is significantly low when model partition is carried out in multi-user scenarios.

D. Resource Allocation Initialization

By analyzing the results of the JM-MPRA algorithm, it is found that the resource allocation scheme with the latency of all devices is usually as equal as possible, or within a certain difference range closer to the final optimal resource allocation scheme. Therefore, we propose a Resource Allocation Vector Initialization (RAVI) algorithm executed in the cloud to replace Step 1 in JM-MPRA algorithm so as to accelerate the converge efficiency of JM-MPRA algorithm. RAVI algorithm is shown in Algorithm 3.

In Algorithm 3, according to the number of devices, the cloud computing resources and communication resources are evenly allocated to initialize the vector as the algorithm input. The values of threshold λ, resource adjustment step size μ_B and μ_C can be set based on the device situation and requirements. The output of the algorithm is a resource allocation vector which makes the latency of each u_i as equal as possible. After resource allocation initialization, the total inference latency of each u_i can be obtained, and T^{arr} represents the total inference latency vector of each u_i. Then RAVI algorithm selects the devices with the minimum and maximum inference latencies, and labels them as i_{min} and i_{max}, respectively. Further, RAVI algorithm continuously makes coarse-grained adjustments of resources until the difference between maximum latency and minimum latency is less than the λ, or when the computing or communication resources allocated to u_i are less than the specified resource adjustment step size.

RAVI algorithm initializes the resource allocation vector reasonably, so that JM-MPRA algorithm can converge more quickly, accelerate the decision-making process, and improve the efficiency of collaborative inference in multi-user scenarios.

VI. Experiment and Evaluation

In this section, we first introduce the experimental setup and comparison methods. Then, we compare RAP with the state-of-the-art methods in different network states. We evaluate the performance of RAP with JM-MPRA algorithm in multi-user scenarios, and we also verify the effectiveness of RAVI in the performance improvement of JM-MPRA.

A. Experimental Setup

We use the Raspberry Pi 4B platform as the end device, which is equipped with a 4-core ARM Cortex-A72@1.5 GHz processor and 4G RAM. For the cloud, we use a server equipped with an Intel Xeon Silver 4212@2.2 GHz processor and an NVIDIA RTX 2080Ti GPU. The framework and algorithms proposed in this paper are implemented with Python, and both end devices and the cloud use PyTorch as their machine-learning engine to execute DNN inferences.

For the datasets, two commonly used datasets with different input sizes for different tasks are adopted, i.e., CIFAR-10[1] (32×32) and ImageNet[2] (224×224). We also consider different DNNs with different architectures. For the chain topology DNN, we use AlexNet [41] and VGG-16 [42]. For the graph-structured DNN, we use GoogLeNet [27], ResNet-50 [28] and Inception-V3 [2]. We pre-train these DNNs on the aforementioned datasets and quantify these models to 8-bit. Then we deploy the quantized DNN models on the end device and the original unquantized models on the cloud.

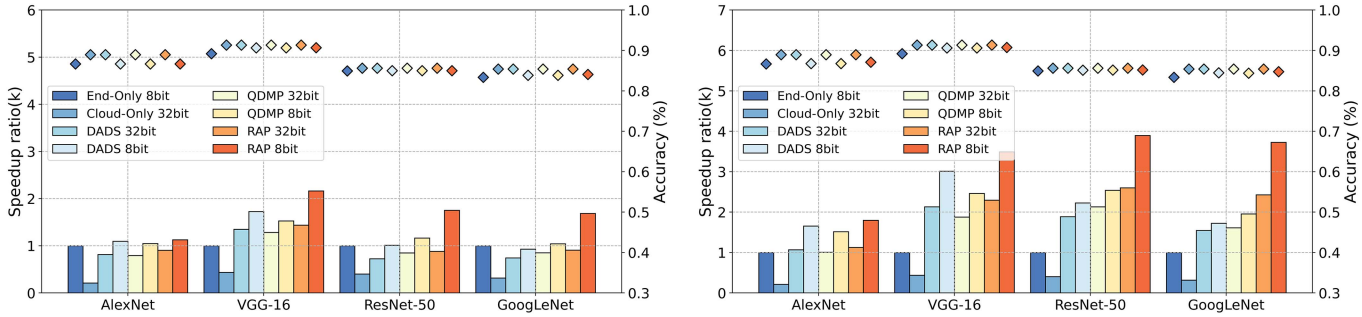
We compare our method with the following methods:

- End-Only: All the DNN inference tasks are executed on the end device.
- Cloud-Only: The raw input data is first transmitted to the cloud, then all the DNN inference tasks are executed on the cloud.
- DADS [11]: This method constructs the entire DNN model as a DAG graph which exhibits high time complexity. And it only considers the optimal inference latency without other optimal objectives when making the model partition decision.
- QDMP [29]: The QDMP method speeds up the decision-making process by finding cut points on the basis of DAG. However, it also doesn't take into account the inference accuracy in the partition.

B. Performance Evaluations in Stable Networks

We first compare the performance of RAP with other methods in stable networks of specific quality and with a quantization bit of 8. Two communication technologies, i.e., 3G and 4G,

[1]https://www.cs.toronto.edu/~kriz/cifar.html
[2]https://www.image-net.org/



(a) 3G Latency (b) 4G Latency

Fig. 10. The performance of different methods on different DNNs in 3G and 4G networks.

TABLE II
THE DECISION TIME (MS) COMPARISON WITH DIFFERENT METHODS

DNN Model	Decision Time(ms)		
	DADS	QDMP	RAP
AlexNet	10.82	24.05	**5.47**
VGG-16	17.82	43.43	**6.89**
ResNet-50	153.78	64.28	**11.36**
GoogLeNet	200.21	83.63	**15.91**

The best results are in bold.

are used for the experiments. The theoretical maximum uplink bandwidths are 1.1 Mbps and 5.85 Mbps, respectively.

1) Latency: Total latency of completing a task consists of local inference latency, transmission latency and decision time for selecting the optimal split point.

In the total latency, split point decision-making time is the most important factor, especially in the mobile environment. Therefore, we specifically compared RAP with other methods in terms of decision time in 3G network. RAP generates all potential split points during the offline stage, and the decision time of RAP only happens online with the complexity of $\mathcal{O}(N)$, while other methods need to search for all layers or reconstruct the DAG according to bandwidth changes. It can be observed from Table II that RAP significantly reduces decision time, especially for complex DNN models. Compared with DADS and QDMP, RAP achieves up to average 13.54x and 5.66x decision time reduction, respectively. For chain topology DNNs, since each vertex is a cut point, in the QDMP method, it is necessary to construct subgraphs between all adjacent vertices, resulting in a decision time that may be longer than in the DADS method.

We then compare the total latency of different methods by evaluating the speedup ratio in 3G and 4G networks, respectively. The latency of End-Only is used as the benchmark. The speedup ratio is defined as:

$$k = \frac{L_{End-Only}}{L_{method}} \tag{11}$$

where $L_{End-Only}$ represents the total latency of the End-Only method, and L_{method} represents the total latency of other methods, i.e., Cloud-only, DADS, QDMP and RAP.

As shown in Fig. 10(a), when the transmission bandwidth is lower(3G), Cloud-Only method has the largest latency due to the significant data transmission latency. Compared with End-Only and Cloud-Only, the speedup ratios of the other three methods all have greatly improved, and RAP has the most obvious improvement which can achieve 2.16x speedup compared with End-Only method. Besides, with faster decision-making, RAP can have a lower total latency and the speedup ratio can be up to 1.82x compared to DADS and QDMP.

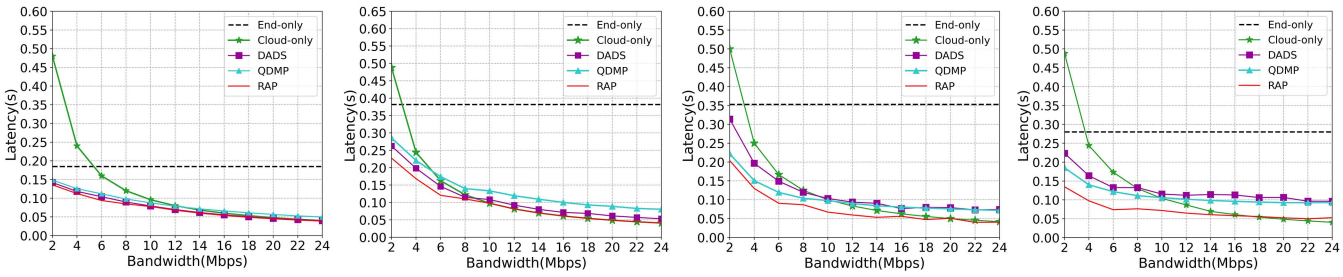
In the case of 4G network, with a higher bandwidth, the data transmission latency is no longer the main factor affecting the total latency. Consequently, in Fig. 10(b), the Cloud-Only method performs better than the End-Only method. Due to the limited computing capacity of the end device, the speedup ratio of RAP can be up to 3.73x compared to End-Only. The total latency of RAP may not be optimal compared to DADS and QDMP, but the faster decision-making still makes RAP have better performance and achieve 2.16x speedup.

2) Accuracy: When comparing the inference accuracy, End-Only method executes the entire quantized DNN on the end device with the largest quantization loss. While Cloud-Only method executes the original unquantified DNN without quantization loss. Other methods use models quantified to 8-bit.

In 3G network, Fig. 10(a) demonstrates that RAP improves accuracy by 0.7%–1.47% compared with End-Only, and 0.19% compared with DADS and QDMP. Due to the low bandwidth, DADS and QDMP select the layer close to the output layer as the split point. This layer has a small output data size resulting in a smaller transmission latency. However, in this situation, more quantized layers will be executed on the end device, leading to a greater quantization loss.

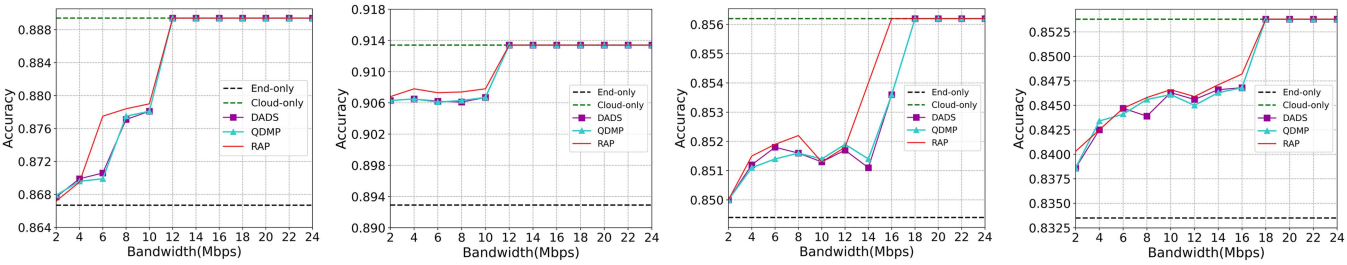
In 4G network, as shown in Fig. 10(b), RAP improves the accuracy by 1.52% compared with End-Only and 0.38% compared with DADS and QDMP. Since the increase of bandwidth, the weight of accuracy is increased. RAP selects the split point closer to the input layer than other methods. Then few layers are executed on the end device to reduce the quantization loss.

3) Quantization: In this section, we demonstrate the influence of quantization on latency and accuracy. In fact, the DADS and QDMP methods did not take model quantization into account. To explore the effect of quantization on latency and accuracy, we deployed 32-bit full precision models and 8-bit



(a) AlexNet (b) VGG-16 (c) ResNet-50 (d) GoogLeNet

Fig. 11. The comparison of latency on different DNNs under different bandwidths.



(a) AlexNet (b) VGG-16 (c) ResNet-50 (d) GoogLeNet

Fig. 12. The comparison of accuracy on different DNNs under different bandwidths.

models separately at the end device in these three methods. In the 3G network, as shown in Fig. 10(a), the speedup ratio of RAP with 8-bit can be up to 1.39x and 1.37x compared to DADS 32-bit and QDMP 32-bit in ResNet-50, with a smaller accuracy reduction of 0.62%, respectively. Since methods deployed with 32-bit models on the end run in full precision, their accuracies are the same as the Cloud-Only solution. For RAP, the comparison effect before and after quantification is more significant. The speedup ratio of RAP can be up to 1.99x compared to 32-bit in ResNet-50. The same phenomenon also exists in 4G networks.

C. Performance Evaluations in Unstable Networks

An unstable network means that the network bandwidth fluctuates frequently, which usually happens in mobile environments. In this section, we evaluate how the unstable network bandwidth affects the model partition.

1) Average Latency and Accuracy Under Different Bandwidth: As shown in Fig. 11, with the increase of bandwidth, the latencies of these four different DNN models gradually decrease.

Under lower bandwidth, RAP mainly optimizes the latency, which has been improved up to 3.53x compared with Cloud-Only in Fig. 11(a), and improved up to 1.72x compared with End-Only in Fig. 11(c). Due to the lower decision-making time, the total inference latency of RAP is lower than other methods.

Under higher bandwidths, RAP gradually increases the consideration of inference accuracy. Therefore, RAP tends to split DNN model at the early layer and let most part of the DNN model execute in the cloud. The inference accuracy of RAP is higher than that of other methods in most cases, as shown in Fig. 12.

2) Split Decision Trigger Times: When the network bandwidth changes dynamically, the OSPS algorithm will be frequently triggered to find the latest optimal split point. In order to alleviate the latency caused by frequent computation, we restrict that only when the bandwidth change exceeds a given threshold, the reselection of the split point will be triggered. From Table IV, we can see the times that the OSPS algorithm is triggered under different given thresholds of bandwidth changes. If the given threshold is too small, the OSPS algorithm will be triggered too frequently, which will lead to instability of the RAP framework and increase the computation. If the given threshold is too large, it will not reselect the optimal split point in time according to the real-time bandwidth. We choose 0.5 Mbps as the given threshold for the experiment, which can not only avoid the frequent triggering of the decision algorithm but also reselect the optimal split point dynamically in time.

3) Average Latency and Accuracy Under Real Network Bandwidth: We first use the bandwidth traces provided in [43] to simulate the changing network bandwidth in urban traffic. Each bandwidth trace in the dataset ranges from 0.8 Mbps to 8 Mbps. Fig. 13 shows an example of the bandwidth trace on the Belgium 4G-LTE dataset. Approximately 600 tasks are executed with the change of network. We conduct experiments on the average latency and accuracy with this bandwidth dataset.

Due to fluctuations in network bandwidth, the decision-making time of different methods may also fluctuate to a certain extent. Nonetheless, in Table III, the decision time of RAP is significantly smaller than DADS and QDMP. The total inference

TABLE III
THE AVERAGE LATENCY (S) AND AVERAGE INFERENCE ACCURACY OF DIFFERENT DNNS ON THE BELGIUM 4G/LTE DATASET

DNN Model	Decision Time(s)			Infer-Trans Latency(s)			Total Latency(s)			Inference Accuracy		
	DADS	QDMP	RAP	DADS	QDMP	RAP	DADS	QDMP	RAP	DADS	QDMP	RAP
AlexNet	0.011	0.019	**0.004**	**0.111**	**0.111**	0.112	0.122	0.130	**0.116**	0.869	0.869	**0.871**
VGG-16	0.020	0.045	**0.013**	0.140	**0.139**	0.190	**0.160**	0.184	0.203	0.906	0.905	**0.907**
ResNet-50	0.103	0.053	**0.016**	**0.135**	**0.135**	0.151	0.238	0.188	**0.167**	0.851	**0.852**	**0.852**
GoogLeNet	0.106	0.087	**0.015**	**0.088**	**0.088**	0.118	0.194	0.175	**0.133**	0.843	0.843	**0.844**

The best results are in bold.

TABLE IV
THE TIMES OF TRIGGERING DECISIONS UNDER DIFFERENT BANDWIDTH CHANGING THRESHOLD

Threshold(Mbps)	0.1	0.5	1.0	1.5	2.0
Times	55	32	18	9	3

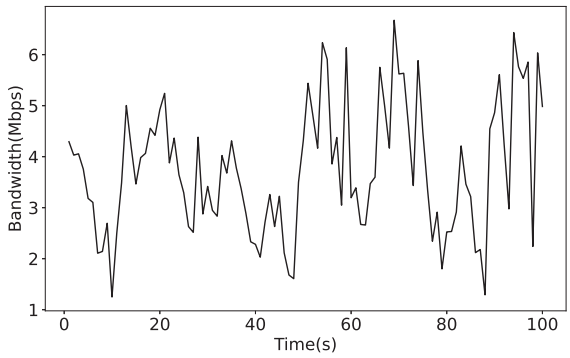


Fig. 13. A bandwidth trace example on the Belgium 4G/LTE dataset.

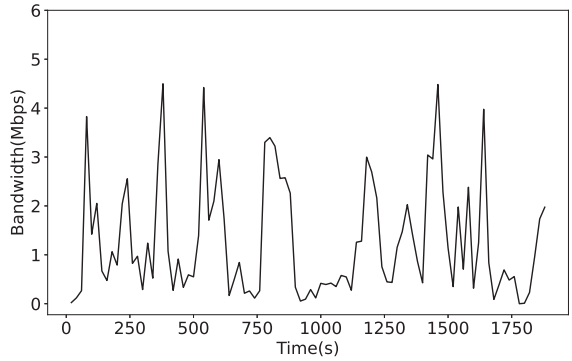


Fig. 14. Railway mobile network bandwidth data.

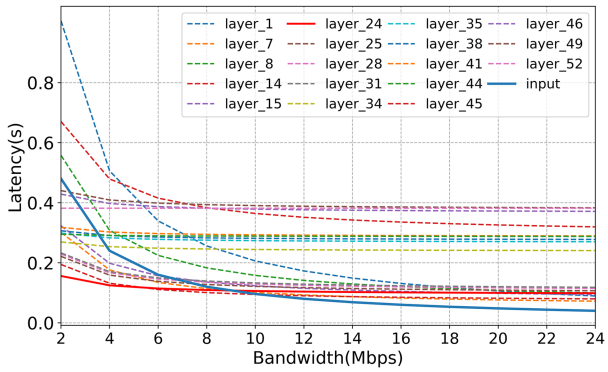


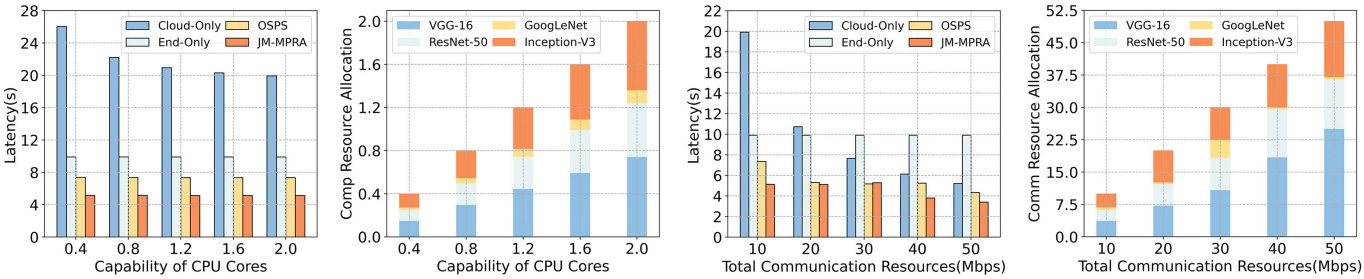
Fig. 15. The inference latency of different split layers on VGG-16 under different network quality.

latency of RAP may not be the best. This is because RAP considers deploying a quantized DNN model on a resource-constrained end device and jointly optimizes total inference latency and the quantization loss. Therefore, the split point may not be the latency optimal split point but is the joint optimal split point. Nevertheless, the fast decision speed can compensate for the total latency. Compared with DADS and QDMP, RAP reduces total latency by up to 3.37x and 1.43x respectively on the four models and improves the accuracy by up to 0.2% when the network quality changes dynamically.

4) The Impact of Network Quality on Model Split Decision: We also construct a bandwidth dataset of the rail mobile network [44]. We obtained the data in the same direction from the same railway line and gathered bandwidth data every 10 seconds. Fig. 14 is a fragment of the bandwidth measurements in a high-speed rail mobile environment. The bandwidth exhibits fluctuations within the range of 0 Mbps to 5 Mbps, with frequent variations and occasional oscillations around 0 Mbps, indicating poor network conditions.

Fig. 15 shows the total inference latency of each 19 potential split point layers on 8-bit VGG-16 under the framework of RAP. When the bandwidth is low, it is better for the VGG-16 model to split at the layer_24 layer to minimize the total inference latency. When the bandwidth is higher, the VGG-16 model can split at the input layer to get the lowest latency. Different split points of the model result in different latency. Therefore, in the dynamically changing network, it is particularly important to make appropriate model split decisions.

We execute different DNN models under the framework of RAP using our constructed bandwidth dataset. In Table V, total layers mean the number of all the potential split point layers, while split layer means the optimal split layer. As shown in Table V, due to the low bandwidth along the high-speed rail, simple DNN models tend to run on the end device rather than uploading data to the cloud, leading to a lower rate of latency speedup; When the model is more complex, running it on the end may be difficult to meet the required resources, resulting in excessive latency. In this case, the RAP framework performs

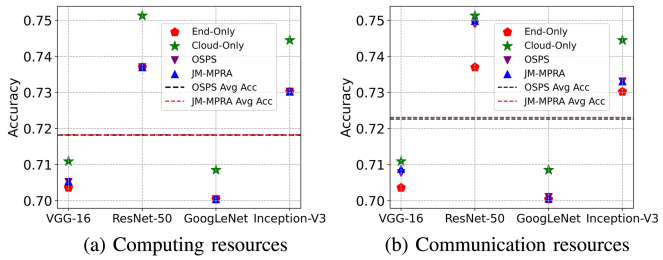


(a) The comparison of latency between different computing resources (b) Computing resources allocation (c) The comparison of latency between different communication resources (d) Communication resources allocation

Fig. 16. Influence of total resources on latency and resources allocation proportion of different DNNs.

TABLE V
SPLIT DECISIONS AND INFERENCE SPEEDUP RATIO IN THE REAL RAILWAY NETWORK BANDWIDTH DATASET

DNN Model	Railway Network		4G-LTE Network	
	Split/Total (Layer)	Speedup Ratio	Split/Total (Layer)	Speedup Ratio
AlexNet	10/10	0	7/10	1.58
VGG-16	14/19	1.32	4/19	1.90
ResNet-50	14/23	1.46	15/23	2.12
GoogLeNet	15/23	1.47	11/23	2.10



(a) Computing resources (b) Communication resources

Fig. 17. Influence of total resources on accuracy of different DNNs.

model partition, which has 1.32x-2.12x latency acceleration compared with End-Only. When the network quality is better, such as 4G network, the intermediate data transmission latency is smaller, RAP makes more layers to execute on the cloud, aiming to reduce the latency.

D. Performance Evaluations in Multi-User Scenarios

In multi-user scenarios, we compare the proposed JM-MPRA method with End-Only, Cloud-Only, and OSPS methods which doesn't consider resource allocation. As for the DNN models, we use VGG-16, ResNet-50, GoogLeNet and Inception-V3.

1) Influence of Resources on Latency and Accuracy: We mainly consider computing resources in the cloud and communication resources(network bandwidth) which are closely associated with DNN inference. We use four user devices and each device is deployed with a different model.

Computing Resource: The total computing resources in the cloud are represented by the available computational capability with the number of CPU cores. We set different CPU cores allocated to the cloud as different computing resources to evaluate the multi-user latency and resource allocation. We fixed the communication resources as 10 Mbps. From Fig. 16(a), we can see that compared with other methods, JM-MPRA algorithm can reduce the total latency for all users by 1.43x-5.06x. That is because JM-MPRA makes more rational utilization of resources among users. As Fig. 16(b) shows that JM-MPRA allocates more computing resources to models which require more computational power. Cloud-Only and OSPS methods have no consideration for multi-user resource allocation (by default is to evenly allocate resources). They can't make full use of resources, resulting in a large latency. Fig. 17(a) is the comparison of the average inference accuracy under different computing resources. Since JM-MPRA considers resource requirements for each model, models with high latency can be offloaded more layers to the cloud for better inference accuracy than End-Only. While models with low latency tend to be executed locally.

Communication Resource: The total communication resources used to transmit data are represented by bandwidths. In this experiment, we compare the influence of communication resources and fix the total computing resources of the cloud as 2 CPU cores. As shown in Fig. 16(c), with the increase of the communication resources, the data transmission latency between end devices and the cloud decreases, thus, the total latency decreases. Compared with other methods, JM-MPRA improves the inference efficiency for all users to 1.04x-3.88x. OSPS can't dynamically adjust communication resources for users (by default is to evenly allocate a fixed bandwidth to each user), so it can't decrease the latency by efficiently utilizing the communication resources according to each user model. In contrast, JM-MPRA algorithm can balance the extra resources of one user device to other devices to maintain the optimal latency. As Fig. 16(d) shows that JM-MPRA allocates more communication resources to models which require more transmission bandwidth. This enables models to offload more layers to the cloud for execution. In Fig. 17(b), JM-MPRA algorithm can improve the average accuracy by 0.05% compared with OSPS(the red and black dashed lines).

2) Influence of RAVI Algorithm on Convergence Time: We compare the JM-MPRA algorithm combined with RAVI algorithm (JM-MPRA+RAVI), the average resource initialization (JM-MPRA+AVG), and random initialization (JM-MPRA+RD)

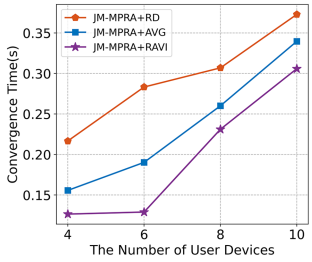


Fig. 18. Influence of the number of user devices on RAVI effectiveness.

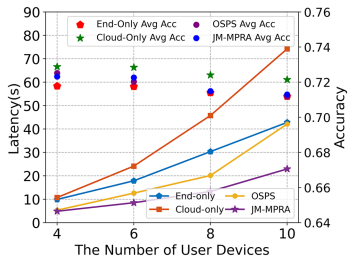


Fig. 19. Influence of the number of user devices on latency and accuracy.

methods in terms of convergence time for making the resource allocation decisions. The only difference between these three methods is the initialized proportion of resource allocation. With RAVI, JM-MPRA can initially adjust resources to make the latency of each device as close as possible, which will be helpful for algorithm convergence in less time. In contrast, JM-MPRA+AVG and JM-MPRA+RD need more iterations to achieve the optimal resource allocation scheme, resulting in a large convergence time. In Fig. 18, with the increasing number of users, the convergence time of these methods also increases. However, it is observed that the JM-MPRA+RAVI algorithm can significantly decrease the convergence time by 1.11x-2.2x compared with the other two methods. This reduction not only demonstrates the efficiency of the JM-MPRA+RAVI combination but also highlights its role in optimizing the JM-MPRA algorithm's overall performance. In the following experiments, for simplicity, we use JM-MPRA to represent JM-MPRA+RAVI.

3) Influence of the Number of User Devices on Latency and Accuracy: We set the total computing resources and communication resources to 2 CPU cores and 20 Mbps, respectively. When the number of user devices is 4, each user device deploys one of the four different DNN models mentioned in Section VI-A. For cases where there are more than 4 users, users after the 5th are configured with randomly selected models with high resource usage rates(like VGG-16 and Inception-V3 in Fig. 16(b) and (d)). As depicted in Fig. 19, JM-MPRA method has a better performance on latency and accuracy than other methods, especially in the scenarios of multiple users with high latency models. Remarkably, compared with the End-Only and Cloud-Only methods, the decrease in total users' latency of JM-MPRA is up to 1.87x and 3.24x respectively. With the number of user devices grows, the Cloud-Only method suffers from higher latency due to the reduced resource allocation for each user device. When the number of user devices is large, there will be more models with high resource usage rates, then the quantization loss for the End-Only method will be high. The improvement of average user's accuracy of JM-MPRA is up to 0.54% compared with End-Only method, when the number of user devices is 4. Compared with OSPS method, the JM-MPRA method can reallocate resources from one user device to others, thereby achieving a lower latency and a higher average accuracy. When the number of user devices increases from 4 to 10, the latency of JM-MPRA decreases from 1.09x to 1.84x and the accuracy improves from 0.1% to 0.24%. In contrast, OSPS method will make a greater proportion of user devices execute locally, resulting in higher latency and lower accuracy. All in all, in multi-user scenarios, JM-MPRA algorithm is better than other methods in reducing latency and enhancing accuracy, especially when the number of devices is large.

VII. Conclusion

In this paper, we propose RAP, a real-time adaptive partition framework for end-cloud collaboration. In different mobile networks, RAP jointly optimizes the inference accuracy and latency, and can obtain optimal model partition decisions with the time complexity of $\mathcal{O}(N)$. Based on RAP framework, we further propose JM-MPRA algorithm to jointly optimize model partition and resource allocation for multiple users. By combining with an initial resource allocation algorithm(RAVI), the JM-MPRA algorithm can converge rapidly. Experiments are conducted on stable and unstable mobile networks. The results show that RAP and JM-MPRA together outperform state-of-the-art methods by up to 5.06x in latency reduction and 1.52% in accuracy improvement. In the future, we will further consider energy consumption in end devices. In addition, since not all layers of a DNN model are equally sensitive to quantization, we will consider using mixed precision quantization instead of quantizing all layers to a similar bit width.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[2] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
[3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 580–587.
[4] R. T. Azuma, "A survey of augmented reality," *Presence*, vol. 6, no. 4, pp. 355–385, 1997.
[5] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
[6] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.
[7] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized convolutional neural networks for mobile devices," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4820–4828.
[8] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, and J. Liu, "Adaptive control of local updating and model compression for efficient federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5675–5689, Oct. 2023.
[9] L. Zhang, Z. Tan, J. Song, J. Chen, C. Bao, and K. Ma, "SCAN: A scalable neural networks framework towards compact and efficient models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4029–4038.

[10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
[11] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1423–1431.
[12] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGPLAN Notices*, vol. 45, no. 1, pp. 615–629, 2017.
[13] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in *Proc. Annu. Int. Conf. Mobile Comput. Netw.*, 2020, pp. 1–15.
[14] C. Ding, A. Zhou, X. Ma, N. Zhang, C.-H. Hsu, and S. Wang, "Towards diversified IoT services in mobile edge computing," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 666–677, First Quarter 2023.
[15] C. Ding, A. Zhou, Y. Liu, R. N. Chang, C.-H. Hsu, and S. Wang, "A cloud-edge collaboration framework for cognitive service," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1489–1499, Third Quarter 2022.
[16] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.
[17] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.
[18] C. Liqun and H. Lei, "Clipping-based neural network post training quantization for object detection," in *Proc. IEEE Int. Conf. Control Electron. Comput. Technol.*, 2023, pp. 1192–1196.
[19] P. Wang, W. Chen, X. He, Q. Chen, Q. Liu, and J. Cheng, "Optimization-based post-training quantization with bit-split and stitching," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2119–2135, Feb. 2023.
[20] R. Chen, L. Li, K. Xue, C. Zhang, M. Pan, and Y. Fang, "Energy efficient federated learning over heterogeneous mobile devices via joint design of weight quantization and wireless transmission," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7451–7465, Dec. 2023.
[21] C. Gong, Y. Lu, K. Xie, Z. Jin, T. Li, and Y. Wang, "Elastic significant bit quantization and acceleration for deep neural networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 3178–3193, Nov. 2022.
[22] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2900–2919, May 2024.
[23] Z. Jiang et al., "Computation and communication efficient federated learning with adaptive model pruning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2003–2021, Mar. 2024.
[24] Z. Li et al., "When object detection meets knowledge distillation: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 10555–10579, Aug. 2023.
[25] J. Gou, X. Xiong, B. Yu, L. Du, Y. Zhan, and D. Tao, "Multi-target knowledge distillation via student self-reflection," *Int. J. Comput. Vis.*, vol. 131, no. 7, pp. 1857–1874, 2023.
[26] M. Poyser and T. P. Breckon, "Neural architecture search: A contemporary literature review for computer vision applications," *Pattern Recognit.*, vol. 147, 2024, Art. no. 110052.
[27] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1–9.
[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[29] S. Zhang et al., "Towards real-time cooperative deep inference over the cloud and edge end devices," *Proc. ACM Interact. Mobile Wearable Ubiquitous Technol.*, vol. 4, no. 2, pp. 1–24, 2020.
[30] A. Banitalebi-Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, and Y. Zhang, "Auto-split: A general framework of collaborative edge-cloud AI," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 2543–2553.
[31] J.-A. Lim and Y. Kim, "Real-time DNN model partitioning for QoE enhancement in mobile vision applications," in *Proc. 19th Annu. IEEE Int. Conf. Sens. Commun. Netw.*, 2022, pp. 407–415.
[32] N. Irtija, I. Anagnostopoulos, G. Zervakis, E. E. Tsiropoulou, H. Amrouch, and J. Henkel, "Energy efficient edge computing enabled by satisfaction games and approximate computing," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 281–294, Mar. 2022.
[33] H. Li, X. Li, Q. Fan, Q. He, X. Wang, and V. C. M. Leung, "Distributed DNN inference with fine-grained model partitioning in mobile edge computing networks," *IEEE Trans. Mobile Comput.*, early access, Jan. 24, 2024, doi: 10.1109/TMC.2024.3357874.
[34] B. Dai, J. Niu, T. Ren, and M. Atiquzzaman, "Toward mobility-aware computation offloading and resource allocation in end–edge–cloud orchestrated computing," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 19450–19462, Oct. 2022.
[35] J. Huang, J. Wan, B. Lv, Q. Ye, and Y. Chen, "Joint computation offloading and resource allocation for edge-cloud collaboration in Internet of Vehicles via deep reinforcement learning," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2500–2511, Jun. 2023.
[36] T. Tan, M. Zhao, and Z. Zeng, "Joint offloading and resource allocation based on UAV-assisted mobile edge computing," *ACM Trans. Sensor Netw.*, vol. 18, no. 3, pp. 1–21, 2022.
[37] X. Tang, X. Chen, L. Zeng, S. Yu, and L. Chen, "Joint multiuser DNN partitioning and computational resource allocation for collaborative edge intelligence," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9511–9522, Jun. 2021.
[38] Y. Kim, J. Kwak, and S. Chong, "Dual-side optimization for cost-delay tradeoff in mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 2, pp. 1765–1781, Feb. 2018.
[39] S. Li, N. Zhang, R. Jiang, Z. Zhou, F. Zheng, and G. Yang, "Joint task offloading and resource allocation in mobile edge computing with energy harvesting," *J. Cloud Comput.*, vol. 11, no. 1, pp. 1–14, 2022.
[40] R. P. Brent, *Algorithms for Minimization Without Derivatives*. Chelmsford, MA, USA: Courier Corporation, 2013.
[41] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
[42] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
[43] J. Van Der Hooft et al., "HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks," *IEEE Commun. Lett.*, vol. 20, no. 11, pp. 2177–2180, Nov. 2016.
[44] X. He, Z. Liu, Z. Kou, N. Dong, Y. Li, and Z. Liu, "HSMS-ADP: Adaptive DNNs partitioning for end-edge collaborative inference in high-speed mobile scenarios," in *Proc. 29th IEEE Int. Conf. Parallel Distrib. Syst.*, 2023, pp. 17–21.

Yiran Li received the BS degree in computer science and technology from the Hebei University of Technology, Tianjin, China, in 2023. She is currently working toward the master degree with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing, China. Her research interests include mobile edge computing, edge intelligence, and cloud computing.

Zhen Liu (Member, IEEE) received the PhD degree in computer architecture from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2006. She is currently a professor with the School of Computer and Information Technology, Beijing Jiaotong University. She is also with the Engineering Research Center of Network Management Technology for High-Speed Railway, Ministry of Education. She has published more than 40 research papers including top journals and conferences, like *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Industrial Informatics*. Her current research interests include intelligent transportation systems, edge intelligence, and edge-cloud collaborative computing.

Ze Kou received the BS degree in computer science from Qingdao University, Shandong, China, in 2020, and the MS degree from the School of Computer and Information Technology, Beijing Jiaotong University, China, in 2023. His research interests include edge computing and cloud computing.

Yannan Wang received the BS degree in traffic engineering from the Civil Aviation University of China, Tianjin, China, in 2019, and the MS degree in control science and engineering from the North China University of Technology, Beijing, China, in 2023. He is currently working toward the PhD degree with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing. His research interests include edge computing, computation offloading, and machine learning.

Guoqiang Zhang received the BS degree in computer science from Nanjing University, Jiangsu, China, in 2002, and the PhD degree in computer architecture from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2008. He is currently a professor with the School of Information, Hainan Normal University. His research interests include future network architecture and network traffic optimization. He has authored or co-authored more than 50 papers in high-level computer science journals and conferences, including the *Science China-Information Sciences*, *Computer Networks*, *Computer Communications*, *IEEE Communications Letters*, *Transactions on Emerging Telecommunications Technologies*, ICC, Globecom, etc. He has also been in charge of a number of national funds and projects about communication and networks.

Yidong Li (Senior Member, IEEE) received the BS degree in electrical and electronic engineering from Beijing Jiaotong University, Beijing, China, in 2003, and the MS and PhD degrees in computer science from the University of Adelaide, Australia, in 2006 and 2010, respectively. He is currently a professor with the School of Computer and Information Technology, Beijing Jiaotong University. He is also the director of the Key Laboratory of Big Data & Artificial Intelligence in Transportation, Ministry of Education. His research interests include Big Data analysis, intelligent transportation, and information security. He has published more than 150 research papers in various journals and refereed conferences. He has organized several international conferences and workshops and has also served as a program committee member for several major international conferences.

Yongqi Sun received the PhD degree in computer science from the Dalian University of Technology, Liaoning, China, in 2006. Currently, he is a professor with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing, China. He is also with the Key Laboratory of Big Data & Artificial Intelligence in Transportation, Ministry of Education. He is also the director with the Institute of Cloud Computing and Data Sciences. His research interests include distributed computing, cloud computing, and data mining. He has published more than 50 research papers.